Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os
# Adjust the path to match your folder location in Drive
os.chdir('/content/drive/MyDrive/AI/Portfolio/GenAI-DeepResearch')
```

Install packages

```
!pip install -r requirements.txt
```

```
Requirement already satisfied: huggingface-hub in /usr/local/lib/python3.13/dist-packages (from -r requirements.txt (line 2)) (1.28.0)
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (from -r requirements.txt (line 3)) (2.32.4)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.13/dist-packages (from -r requirements.txt (line 4)) (1.2.3)
Requirement already satisfied: diskcache in /usr/local/lib/python3.13/dist-packages (from -r requirements.txt (line 5)) (5.6.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (from -r requirements.txt (line 6)) (2.1.3)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from -r requirements.txt (line 7)) (3.1.6)
Requirement already satisfied: pydantic in /usr/local/lib/python3.13/dist-packages (from -r requirements.txt (line 8)) (2.13.4)
Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt (
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt (lin
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt (lin
Requirement already satisfied: hf-xet<2.0.0,>=1.5.2 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt (lin
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt (line
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt (line 2))
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.txt (line 2)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub->-r requirements.t
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests->-r requirements.txt (li
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests->-r requirements.txt (line 3)) (3.19
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests->-r requirements.txt (line 3))
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests->-r requirements.txt (line 3))
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->-r requirements.txt (line 7)) (3.0
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic->-r requirements.txt (line
Requirement already satisfied: pydantic-core==2.46.4 in /usr/local/lib/python3.13/dist-packages (from pydantic->-r requirements.txt (line
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from pydantic->-r requirements.txt (li
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggingface-hub->-r requirements.t
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggingface-hub->-r requir
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub
```

```
!pip install llama-cpp-python --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cu124

```

```
Looking in indexes: https://pypi.org/simple, https://abetlen.github.io/llama-cpp-python/whl/cu124
Requirement already satisfied: llama-cpp-python in /usr/local/lib/python3.13/dist-packages (0.3.35)
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.13/dist-packages (from llama-cpp-python) (4.16.0)
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.13/dist-packages (from llama-cpp-python) (2.1.3)
Requirement already satisfied: diskcache>=5.6.1 in /usr/local/lib/python3.13/dist-packages (from llama-cpp-python) (5.6.3)
Requirement already satisfied: jinja2>=2.11.3 in /usr/local/lib/python3.13/dist-packages (from llama-cpp-python) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2>=2.11.3->llama-cpp-python) (3.0.3)
```

Load Tokens

Load tokens and verify the tokens.

```
import os
from google.colab import userdata

try:
    os.environ["SERPER_API_KEY"] = userdata.get("SERPER_API_KEY")
    print("Serper API Key loaded successfully.")
except Exception as e:
    print("Could not load secret automatically. Please set it manually: os.environ['SERPER_API_KEY'] = 'your_key'")

try:
    HF_TOKEN = userdata.get("HF_TOKEN")
    print("Hugging Face Token loaded successfully.")
except Exception as e:
    print("Could not load Hugging Face token automatically. Please ensure 'HF_TOKEN' is set in Colab secrets.")
    HF_TOKEN = None # Set to None if not loaded


```

```
Serper API Key loaded successfully.
Hugging Face Token loaded successfully.
```

Download Model

We will now download the Gemma-7B-Instruct model, specifically the `Q4_K_M` quantized version. This version provides a good balance between performance and file size, making it suitable for Colab and managing Google Drive storage.

First, we'll ensure a `models` directory exists, then use `wget` to download the model from Hugging Face.

```
import os

# Define the directory and path for the new model
model_dir = 'models'
new_model_name = 'gemma-7b-it-Q4_K_M.gguf' # Common quantized version name
model_path = os.path.join(model_dir, new_model_name)

# Create the models directory if it doesn't exist
os.makedirs(model_dir, exist_ok=True)

print(f"Model directory: {model_dir}")
print(f"New model path will be: {model_path}")
```

```
Model directory: models
New model path will be: models/gemma-7b-it-Q4_K_M.gguf
```

```
import os
from huggingface_hub import hf_hub_download

repo_id = "mlabonne/gemma-7b-it-GGUF" # Keeping this for now, as it worked for download
original_filename = "gemma-7b-it.Q4_K_M.gguf"
target_filename = 'gemma-7b-it-Q4_K_M.gguf' # This is what test.py expects

os.makedirs(model_dir, exist_ok=True)

# Full path where the model will be stored (after potential renaming)
model_path_original = os.path.join(model_dir, original_filename)
model_path_target = os.path.join(model_dir, target_filename)

# Download only if the target filename doesn't already exist
if not os.path.exists(model_path_target):
    print(f"Checking for {target_filename}...")
    if not os.path.exists(model_path_original): # If original named file doesn't exist either, download
        print(f"Downloading {original_filename} from {repo_id}...")
        try:
            downloaded_path = hf_hub_download(
                repo_id=repo_id,
                filename=original_filename,
                local_dir=model_dir,
                token=HF_TOKEN
            )
            print(f"Download complete: {downloaded_path}")
        except Exception as e:
            print(f"Error downloading model: {e}")
            model_path = None # Set to None if download fails
            print(f"Model path set to: {model_path}")
            raise # Re-raise the exception to stop further execution if download fails
    else:
        print(f"Found {original_filename}. Proceeding to rename.")

    # Rename the file if the original exists and the target does not
    if os.path.exists(model_path_original) and not os.path.exists(model_path_target):
        try:
            os.rename(model_path_original, model_path_target)
            print(f"Renamed '{original_filename}' to '{target_filename}'.")
        except Exception as e:
            print(f"Error renaming file: {e}")
            # If rename fails, we still point to original if it exists
            model_path = model_path_original
    elif os.path.exists(model_path_target):
        print(f"Target file '{target_filename}' already exists.")

    model_path = model_path_target # Update model_path to the target name

else:
    print(f"Model already exists at: {model_path_target}")
    model_path = model_path_target

print(f"Model path: {model_path}")
```

```
Model already exists at: models/gemma-7b-it-Q4_K_M.gguf
Model path: models/gemma-7b-it-Q4_K_M.gguf
```

```
import os

print(os.path.exists('models/gemma-7b-it-Q4_K_M.gguf'))
print(os.listdir('models'))
```

```
True
['.cache', 'gemma-7b-it-Q4_K_M.gguf']
```

Code components

Research Config

```
import os
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()

@dataclass
class ResearchConfig:
     # Update this path to where your Gemma-7b GGUF file lives
    model_path: str = model_path_target #"models/gemma-7b-it-Q4_K_M.gguf"
    max_tokens: int = 2048
    temperature: float = 0.4  # Lower temperature keeps Gemma highly focused on facts
    top_p: float = 0.95
    top_k: int = 40
    context_length: int = 4096 # Gemma-7b natively supports up to 8k context!
    search_api_key: str = os.getenv("SERPER_API_KEY", "")
    search_engine: str = "serper"
```

Report Config

```

class ReportConfig:
  def get_meta_themes(self)-> list[str]:
    return {
      "EXECUTIVE SUMMARY", "ABSTRACT", "INTRODUCTION",
      "OVERVIEW", "CONCLUSION", "NEXT STEPS"
    }

  def get_standard_themes(self)-> list[str]:
    meta_themes = self.get_meta_themes()
    themes = self.themes_for_format(report_format)
    return [x for x in themes if x not in meta_themes]

  def themes_for_format(self, fmt: str) -> list[str]:
    """This function generates themes tailored to the selected report format (e.
    g., executive, detailed, academic).
    Args:
        fmt (str): The selected report format.

    Returns:
        list[str]: A list of themes for the specified format.
    """
    fmt = (fmt or "").strip().lower()
    if fmt == "executive":
        return ["EXECUTIVE SUMMARY"]
    if fmt == "detailed":
        return [
            "INTRODUCTION",
            "DETAILED ANALYSIS",
            "CURRENT TRENDS AND DEVELOPMENTS",
            "IMPLICATIONS AND RECOMMENDATIONS",
            "CONCLUSION",
        ]
    if fmt == "academic":
        return [
            "ABSTRACT",
            "INTRODUCTION",
            "METHODOLOGY",
            "FINDINGS",
            "DISCUSSION",
            "CONCLUSION",
        ]
    if fmt == "presentation":
        return [
            "OVERVIEW",
            "KEY INSIGHTS",
            "RECOMMENDATIONS",
            "NEXT STEPS",
            "CONCLUSION",
```

```
        ]
    return ["INTRODUCTION", "DETAILED ANALYSIS", "CONCLUSION"]
```

Web Search Manager

```
import os
from llama_cpp import Llama
from typing import List, Dict, Optional
import aiohttp
import re
import asyncio
import time
import json

import requests

class WebSearchManager:
    def search_web(self, query: str, num_retries: int = 3) -> list:
        url = "https://google.serper.dev/search"
        payload = {"q": query, "num": 5}
        headers = {
            "X-API-KEY": os.environ.get("SERPER_API_KEY", ""),
            "Content-Type": "application/json"
        }
        try:
            response = requests.post(url, json=payload, headers=headers,
            timeout=10)
            data = response.json()
            results = []
            for item in data.get('organic', []):
                results.append({
                    'title': item.get('title', ''),
                    'url': item.get('link', ''),
                    'snippet': item.get('snippet', ''),
                    'source': 'web'
                })
            return results
        except Exception as e:
            print(f"Search error for query '{query}': {e}")
            return []

```

LLM Manager

```
import json
import re

class LlmManager:
    def __init__(self, config: ResearchConfig):
        self.config = config
        self.llm = None
        self.demo_mode = False

    def load_model(self):
      try:
          # Validate model file
          print(self.config.model_path)
          if not os.path.exists(self.config.model_path):
              raise FileNotFoundError(
                  f"Model file not found: {self.config.model_path}"
              )

          file_size_gb = os.path.getsize(self.config.model_path) / (1024 ** 3)

          if file_size_gb < 1.0:
              raise ValueError(
                  f"Model file is too small ({file_size_gb:.1f} GB). "
                  "The model file may be incomplete or corrupted."
              )

          # Load model
          self.llm = Llama(
              model_path=self.config.model_path,
              n_ctx=self.config.context_length,
              verbose=False,
              n_gpu_layers=-1,
              use_mmap=True,
              use_mlock=False,
              n_batch=512,
          )

          # Test generation
          test = self.llm(
```

```
            "Hi",
            max_tokens=3,
            temperature=0.1,
            echo=False
        )

        if not test or "choices" not in test:
            raise RuntimeError(
                "Model loaded successfully, but test generation failed."
            )

        return True

    except (FileNotFoundError, ValueError, RuntimeError):
        raise

    except Exception as e:
        raise RuntimeError(
            f"Model loading failed: {e}"
        ) from e

def prompt_llm(self, prompt: str, max_tokens: int = None, extra_stops: Optional[List[str]] = None) -> str:
    if not self.llm:
        return "Model not loaded."

    stops = ["<end_of_turn>", "<eos>"]
    if extra_stops:
        stops.extend(extra_stops)
    mt = max_tokens or self.config.max_tokens

    try:
        resp = self.llm(
            prompt,
            max_tokens=mt,
            temperature=self.config.temperature,
            top_p=self.config.top_p,
            top_k=self.config.top_k,
            stop=stops,
            echo=False
        )
        return resp["choices"][0]["text"].strip()
    except Exception as e:
        return f"Error generating response: {str(e)}"


def parse_llm_json(self, response_text: str):
  if not response_text:
      return None

  # 1. Strip markdown code blocks if present (e.g. ```json ... ```)
  clean_text = re.sub(r"```(?:json)?\s*([\s\S]*?)\s*```", r"\1", response_text).strip()

  # 2. Try direct parsing first
  try:
      return json.loads(clean_text)
  except json.JSONDecodeError:
      pass

  # 3. Fallback: Search for the first '[' or '{' and last ']' or '}'
  try:
      match = re.search(r"(\{[\s\S]*\}|\[[\s\S]*\])", clean_text)
      if match:
          return json.loads(match.group(1))
  except json.JSONDecodeError:
      pass

  return None

def extract_final_block(self, text: str) -> str:
    if not text:
        return ""

    # Strip out common chat model thinking or conversational preambles at the very start
    preamble_patterns = [
        r"^(?:note:|okay,|hmm,|internal|let me|i (?:will|'ll)|as an ai|thinking|plan:|here is your report|the following is|i ha
        r"^(?:Here is the report|I have compiled the report|The report is provided below|This is the requested report).*?$",
        r"^(?:Please find the report below|Here's the report).*?$"
    ]
    for pattern in preamble_patterns:
        cleaned_text = re.sub(pattern, "", cleaned_text, flags=re.IGNORECASE | re.MULTILINE).strip()

    return cleaned_text
```

Deep Research Assistant

```
class DeepResearchAssistant:
  def __init__(self, researchConfig: ResearchConfig, reportConfig: ReportConfig, llmManager: LlmManager, webSearchManager:WebSearch
    self.researchConfig = researchConfig
    self.reportConfig = reportConfig
    self.llmManager = llmManager
    self.webSearchManager = WebSearchManager
    self.global_seen_urls = set()

  def synthesize_research(self, topic: str, report_format: str) -> str:
    # 1. Initialize model on the T4 GPU
    print("\nInitializing model on the T4 GPU...")
    self.llmManager.load_model()

    # 2. Gather inputs
    print("\n--- TEST AGENT RUNNER ---")

    # 3. Get themes
    themes = self.reportConfig.themes_for_format(report_format)
    standard_themes = self.reportConfig.get_standard_themes()
    standard_themes_text = ",".join(standard_themes)

    # --- STEP 1: GENERATE SUB-TOPICS (THE OUTLINE) ---
    sub_topics = self.get_outline(topic, report_format, standard_themes_text)

    # --- STEP 2: MULTI-STEP GENERATION LOOP ---
    final_report = self.synthesize_sections(topic, sub_topics, themes, standard_themes)
    print("\n--- FINAL MULTI-STEP REPORT OUTPUT ---")
    print(final_report)

    return final_report

  def get_outline(self, topic: str, report_format: str, standard_themes_text: str) -> list:
    print("\n[Step 1] Asking model to break down the topic...")
    outline_prompt = f"""<start_of_turn>user
    Topic: {topic}
    Report format: {report_format}
    Suggested themes: {standard_themes_text}
    Tasks:
    1. Identify distinct one or more sub-topics for a professional report on '{topic}, for each of the suggested themes'.
    2. Uner each them, order the sub-topics in a logical hierarchy.
    3. For each sub-topic generate up to 3 concise web-search query suffixes.
    Rules:
    - Return ONLY a raw JSON array. No explanations, no python code blocks, no markdown ticks.
    - Maximum 3 queries per sub-topic.
    - Do NOT include "{topic}" in the queries.
    Output Format:
    [
      {{
        "theme": "DETAILED ANALYSIS",
        "sub_topic": "Subtopic Name Here",
        "queries": [
          "query suffix one",
          "query suffix two"
        ]
      }}
    ]
    <end_of_turn>
    <start_of_turn>model
    """

    response = self.llmManager.prompt_llm(outline_prompt, max_tokens=600)
    print(f"Model output Prompt 1: {response}")

    # Use LlmManager's built-in robust JSON parser
    raw_sub_topics = self.llmManager.parse_llm_json(response)

    # Fallback default sub-topics if parsing fails or returns non-list
    if not isinstance(raw_sub_topics, list) or len(raw_sub_topics) == 0:
        print("Warning: Could not parse valid JSON from LLM outline. Using default fallback outline.")
        raw_sub_topics = [
            {
                "theme": "INTRODUCTION",
                "sub_topic": f"Introduction to {topic}",
                "queries": [f"{topic} overview", f"what is {topic}"]
            },
            {
                "theme": "ANALYSIS",
                "sub_topic": f"Key aspects of {topic}",
                "queries": [f"key features of {topic}", f"{topic} analysis"]
            }
```

```
        ]

    # Deduplication and cleaning logic
    sub_topics = []
    seen_subtopics = set()

    for item in raw_sub_topics:
        name = item.get("sub_topic", "").strip()
        name_lower = name.lower()

        # Skip if this sub-topic was already added (deduplication)
        if name_lower and name_lower not in seen_subtopics:
            seen_subtopics.add(name_lower)

            # Clean and deduplicate queries within the sub-topic
            filtered_queries = []
            seen_queries = set()
            for q in item.get("queries", []):
                full_q = f"{topic} {q}".strip().lower()
                if full_q not in seen_queries:
                    seen_queries.add(full_q)
                    filtered_queries.append(f"{topic} {q}".strip())

            item["queries"] = filtered_queries
            sub_topics.append(item)

    return sub_topics

    print(f"-> Sub-topics chosen: {sub_topics}")

  def synthesize_sections(self, topic: str, sub_topics: list, themes:list, standard_themes:list) -> str:
    print("\n[Step 2] Beginning sequential generation loop...")

    # Main topic header (Level 1)
    final_report = f"# Research Report: {topic}\n\n"
    global_seen_urls = set()

    for theme_name in themes:
      final_report += f"## {theme_name}\n\n"
      print(f"Synthesizing meta-section: {theme_name}...")

      # CASE 1: Handle Meta Sections (No search queries needed)
      if theme_name not in standard_themes:
        final_report = self.synthesize_meta_section(final_report, theme_name)
      else:
        final_report = self.synthesize_standard_section(final_report, theme_name, sub_topics)

        # Display the compiled result
    return final_report

  def synthesize_meta_section(self, final_report:str, theme_name:str) -> str:
    print(f"Synthesizing meta-section: {theme_name}...")

    context_block = ""
    if len(final_report.strip()) > 50:
      context_block = f"\nPrevious Report Draft Content:\n{final_report}\n"

      meta_prompt = f"""<start_of_turn>user
Write a professional, comprehensive paragraph for the section "{theme_name}" regarding the topic "{topic}".
Focus entirely on high-level narrative synthesis and context. Do not use bullet points.
{context_block}
<end_of_turn>
<start_of_turn>model
"""
    content = self.llmManager.prompt_llm(meta_prompt, max_tokens=350)
    return final_report + f"{content}\n\n"

  def synthesize_standard_section(self, final_report:str, theme_name:str, sub_topics:list) -> str:
    print(f"Running research loop for analytical theme: {theme_name}...")

    # Filter sub-topics belonging to this specific theme
    theme_sections = [s for s in sub_topics if s.get("theme", "").strip().upper() == theme_name]

    # Fallback if the model didn't explicitly map a sub-topic to this theme
    if not theme_sections:
        theme_sections = [{"sub_topic": theme_name, "queries": [f"{topic} {theme_name.lower()}"]}]

    for section in theme_sections:
      sub_topic_name = section.get("sub_topic", "General Section").strip()

      print(f"Generating details for [{theme_name}] -> {sub_topic_name}...")

      search_results = []
      for query in section.get("queries", []):
          results = self.webSearchManager.search_web(query, 3)
          for res in results:
              url = res.get("url", "")
              if url and url not in self.global_seen_urls:
```

```
                self.global_seen_urls.add(url)
                search_results.append(res)

    context_lines = []
    for i, result in enumerate(search_results[:20]):
        title = result.get("title", "")
        snippet = result.get("snippet", "")
        context_lines.append(f"Source {i+1} Title: {title}\nSource {i+1} Summary: {snippet}")
    context = "\n".join(context_lines)

    section_prompt = f"""<start_of_turn>user
Write a detailed, comprehensive paragraph explaining the role of "{sub_topic_name}" regarding the broader topic "{topic}", under
Focus entirely on factual insights and given Research Notes. Do not use bullet points or intros.

Research Notes:
{context}
<end_of_turn>
<start_of_turn>model
"""

    section_text = self.llmManager.prompt_llm(section_prompt, max_tokens=400)

    # Sub-topic header (Level 3) nested under the theme
    final_report += f"### {sub_topic_name}\n{section_text}\n\n"
    return final_report
```

Deep Research Report generation

```
researchConfig = ResearchConfig()
reportConfig = ReportConfig()
llmManager = LlmManager(researchConfig)
webSearchManager = WebSearchManager()

deepResearchAssistant = DeepResearchAssistant(researchConfig, reportConfig, llmManager, webSearchManager)

topic = "Latest trends in car designs"
report_format = "detailed"
deepResearchAssistant.synthesize_research(topic, report_format )

```

```
Initializing model on the T4 GPU...
models/gemma-7b-it-Q4_K_M.gguf

--- TEST AGENT RUNNER ---

[Step 1] Asking model to break down the topic...
Model output Prompt 1: [
      {
        "theme": "DETAILED ANALYSIS",
        "sub_topic": "Sustainable car design",
        "queries": ["Sustainable car design trends", "Eco friendly car design", "Future sustainable car design"]
      },
      {
        "theme": "DETAILED ANALYSIS",
        "sub_topic": "Autonomous driving technology",
        "queries": ["Autonomous driving technology advancements", "Self driving car technology", "Future of autonomous driving"]
      },
      {
        "theme": "CURRENT TRENDS AND DEVELOPMENTS",
        "sub_topic": "Electric vehicles",
        "queries": ["Electric car trends", "EV market growth", "Future of electric vehicles"]
      },
      {
        "theme": "CURRENT TRENDS AND DEVELOPMENTS",
        "sub_topic": "Autonomous trucking",
        "queries": ["Autonomous trucking advancements", "Self driving truck technology", "Future of autonomous trucking"]
      },
      {
        "theme": "IMPLICATIONS AND RECOMMENDATIONS",
        "sub_topic": "Impact of new car designs on society",
        "queries": ["Impact of car design on society", "Social impact of new car designs", "Recommendations for car design to improve soc
      },
      {
        "theme": "IMPLICATIONS AND RECOMMENDATIONS",
        "sub_topic": "Future of car design",
        "queries": ["Future trends in car design", "Emerging technologies in car design", "Recommendations for future car design"]
      }
    ]

[Step 2] Beginning sequential generation loop...
Synthesizing meta-section: INTRODUCTION...
Synthesizing meta-section: INTRODUCTION...
Synthesizing meta-section: DETAILED ANALYSIS...
Running research loop for analytical theme: DETAILED ANALYSIS...
Generating details for [DETAILED ANALYSIS] -> Sustainable car design...
Synthesizing meta-section: CURRENT TRENDS AND DEVELOPMENTS...
Running research loop for analytical theme: CURRENT TRENDS AND DEVELOPMENTS...
Generating details for [CURRENT TRENDS AND DEVELOPMENTS] -> Electric vehicles...
Synthesizing meta-section: IMPLICATIONS AND RECOMMENDATIONS...
Running research loop for analytical theme: IMPLICATIONS AND RECOMMENDATIONS...
Generating details for [IMPLICATIONS AND RECOMMENDATIONS] -> Impact of new car designs on society...
Synthesizing meta-section: CONCLUSION...
Synthesizing meta-section: CONCLUSION...

--- FINAL MULTI-STEP REPORT OUTPUT ---
# Research Report: Latest trends in car designs

## INTRODUCTION

The automotive industry is perpetually evolving, with each passing year unveiling new trends and innovations. The latest trends in car de

## DETAILED ANALYSIS

### Sustainable car design
Sustainable car design plays a pivotal role in the latest trends in car designs, shaping the future of transportation towards greener, ec

## CURRENT TRENDS AND DEVELOPMENTS

### Electric vehicles
Electric vehicles (EVs) are revolutionizing the automotive industry, shaping the latest trends in car designs. With their zero tailpipe e

## IMPLICATIONS AND RECOMMENDATIONS

### Impact of new car designs on society
The impact of new car designs on society is multifaceted and profound. From an aesthetic standpoint, new car designs influence fashion tr

## CONCLUSION

The latest trends in car designs are characterized by a confluence of factors, including technological advancements, shifting consumer pr


'# Research Report: Latest trends in car designs\n\n## INTRODUCTION\n\nThe automotive industry is perpetually evolving, with each passin
g year unveiling new trends and innovations. The latest trends in car designs are characterized by a confluence of factors, including te
chnological advancements, shifting consumer preferences, and environmental concerns. One prominent trend is the increasing adoption of a
utonomous driving technologies, leading to the integration of AI-powered systems and sensors into vehicles. Additionally, the rise of el
ectric vehicles has revolutionized the way we travel, with manufacturers offering a wide range of electric and hybrid models to cater to
various budgets and needs. Sustainability has also influenced design trends, with companies incorporating eco-friendly materials and red
ucing overall vehicle emissions. Ultimately, the latest trends in car designs are shaping the future of transportation, offering drivers
```

Streamlit